Project 3: Secret Sharing

Secure Multiparty Computation: Average Computation using Paillier Encryption and Shamir's Secret Sharing

Tren Meckel

October 8, 2025

Abstract very brief & small

This project explores the performance and implementation differences between three methods for computing the average of distributed values in a multiparty setting: a non-private baseline, Paillier homomorphic encryption, and Shamir's Secret Sharing. Each approach ensures varying levels of privacy and computational cost. Runtime results are analyzed to assess scalability with increasing numbers of parties.

Introduction

Secure multiparty computation (SMPC) allows multiple parties to jointly compute a function over their inputs while keeping those inputs private. This experiment focuses on comparing the efficiency of two privacy-preserving techniques—Paillier Encryption and Shamir's Secret Sharing—against a non-private baseline when computing the average of distributed values.

Paillier Encryption provides additive homomorphism, allowing computations to be performed directly on encrypted values. Shamir's Secret Sharing, on the other hand, distributes secrets across multiple participants using polynomial interpolation. Both methods were implemented in Python and evaluated under identical conditions to compare runtime performance.

Methodology

The experiment was conducted with varying numbers of parties $n \in \{10, 50, 100, 250, 500\}$. For each n:

1. Random integer values were generated to simulate each party's input.
2. The non-private average was computed as a baseline.
3. The Paillier encryption scheme was used for secure aggregation of encrypted values.

4. Shamir's Secret Sharing was applied by splitting each party's secret, aggregating shares, and reconstructing the final sum to compute the average.

Each computation's runtime was recorded using Python's `time` module. For Shamir's method, the number of participating parties n also represented the number of shares, with a reconstruction threshold of $t = \lfloor n/2 \rfloor$.

Results

The runtime results are summarized in Table 1 and visualized in Figures 1 and 2. As expected, the non-private computation performed almost instantaneously, while Paillier and Shamir methods showed significant increases in runtime as n grew.

Table 1: Runtime results for each method (in seconds).

Number of Parties (n)	**Non-private**	**Paillier**	**Shamir**
10	0.0001	3.0546	0.0004
50	0.0000	13.5917	0.0367
100	0.0000	26.8457	0.4317
250	0.0001	65.4630	13.2428
500	0.0000	130.0811	212.8965

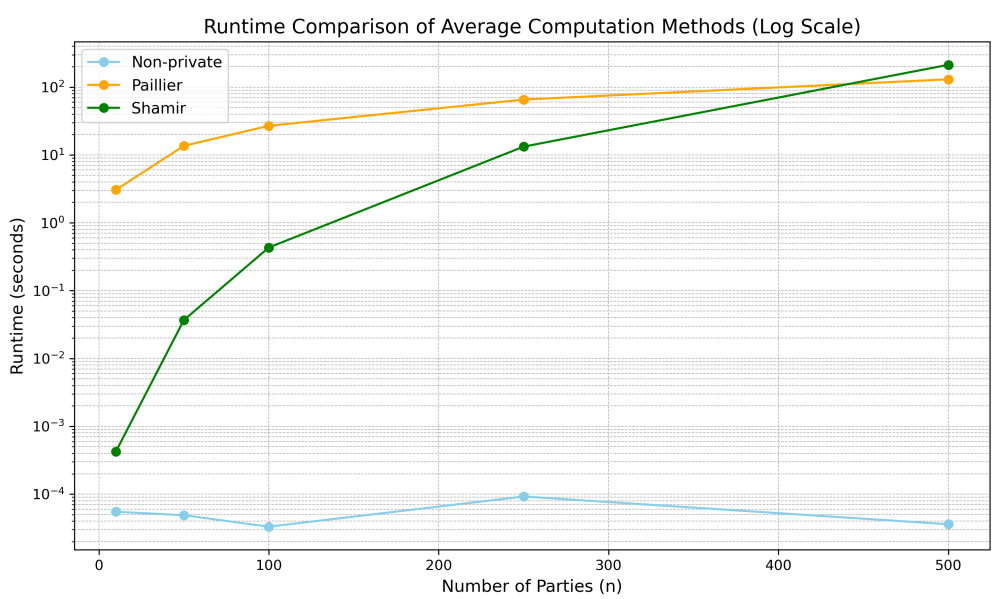

Figure 1: Runtime comparison of all methods (logarithmic scale).

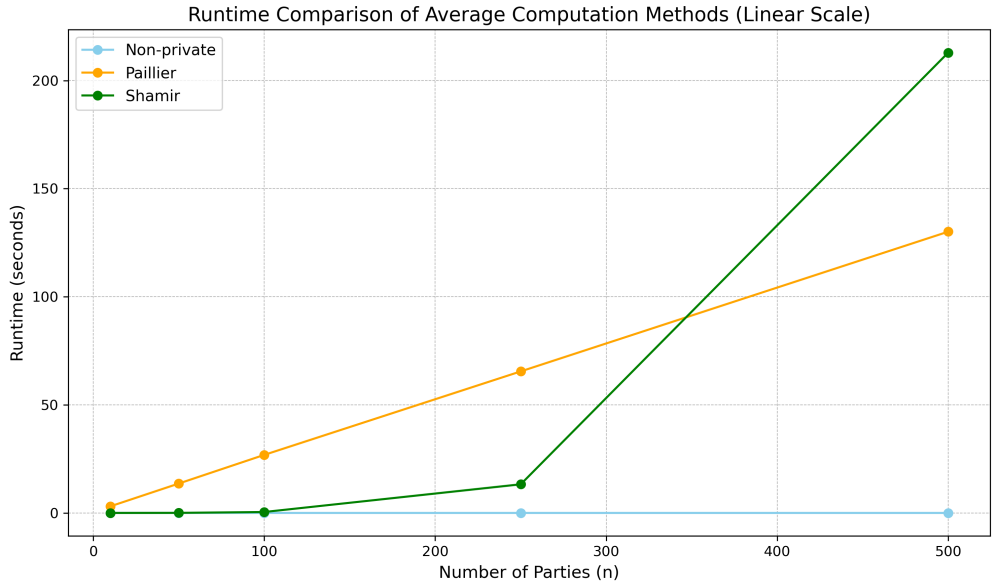

Figure 2: Runtime comparison of all methods (linear scale).

Discussion

The results clearly show a trade-off between privacy and computational efficiency:

- **Non-private:** Executes in negligible time regardless of n since no encryption or sharing is required.
- **Paillier Encryption:** Demonstrates a steady linear-to-superlinear growth in runtime due to costly modular arithmetic and encryption overhead.
- **Shamir's Secret Sharing:** Performs faster than Paillier for smaller n, but runtime escalates significantly as the number of parties increases, particularly due to polynomial evaluations and reconstruction costs.

The logarithmic plot (Figure 1) emphasizes that Paillier's cost grows rapidly but predictably, while Shamir's approach incurs a sharper increase after a threshold, indicating polynomial growth. For small to moderate n, Shamir's method is more practical, while Paillier offers stronger privacy guarantees suitable for high-security applications.

Conclusion

Both Paillier Encryption and Shamir's Secret Sharing successfully enable privacy-preserving average computation but differ drastically in scalability. The experiment demonstrates the inherent computational costs of privacy and highlights how choosing between the two methods depends on the required balance between efficiency and security.